Bài viết báo cáo

1.Tổng quan

- Thông tin cá nhân:Nguyễn Viết Bảo Nguyên (MSV:25022967)
- Các nhóm chức năng đã cài đặt thành công:
  - Game Interaction: `validateInput`, `getInput`, `selectSize`, `selectGoal`, `selectGameMode`, `selectBotLevel`, `getPlayerMove.`
  - Game Renderer: `displayBoard`, `showPlayer`, `showMove`, `showInvalidMove`, `showResult`, `printResult.`
  - Game Logic: `initBoard`, `isValidMove`, `makeMove`, `isEmptyHead`, `checkWin`, `checkDraw.`
  - Game Engine: `startGame`, `playGame`, `endGame.`
  - Bot Module: `random_pick`.
- Cấp độ Bot đã triển khai: EASY

2.Chi tiết tổng quan

1. Game Interaction

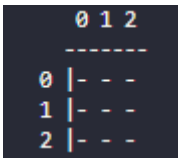
- E đã xử lý đầu vào bằng hàm validateInput,sử dụng 'isdigit()' để kiểm tra xem đầu vào có phải là số hay không,nếu không là số thì sẽ trả về false 'return false' rồi cho hỏi lại.
- Trong trường hợp gửi sai tọa độ ngoài phạm vi,hay là input không phải là số,để ko bị crash:

  Chương trình xử lý input validation qua 2 hàm chính:

  - Hàm `validateInput` kiểm tra từng ký tự trong chuỗi nhập vào bằng `std::isdigit`, nếu phát hiện ký tự không phải số thì trả về `false`.
  - Hàm `getInput` gọi `validateInput`, nếu không hợp lệ thì trả về `false` ngay, không thực hiện chuyển đổi sang số nguyên.

Khi người dùng nhập sai định dạng hoặc tọa độ ngoài phạm vi, các hàm như `selectSize`, `selectGoal`, `getPlayerMove` trả về `false` → Game Engine sử dụng vòng lặp `do...while` để hỏi lại liên tục cho đến khi người dùng nhập đúng, đảm bảo chương trình không bị crash.

2.Game Renderer

- Bàn chơi sẽ có kích thước tùy vào người chơi nhập vào,dòng '----' và dòng '|' dùng để phân tách tiêu đề và nội dung bàn cờ,dòng trên cùng sẽ hiển thị số thứ tự ô trống dọc theo các cột,cột ngoài cùng bên trái sẽ là số thứ tự ô trống theo hàng ngang,để khi đó người chơi có thể nhập vào ô mình muốn theo các tọa độ trên bảng(

đây là bảng 3x3) .Còn về các kí hiệu 'X' và 'O' thì mỗi lượt đi sẽ là 1 người nhập,với Player 1 là kí hiệu 'X',Player 2 là kí hiệu 'O',với những ô còn trống thì sẽ là kí hiệu '-'

3.Game Logic

- Pseudo-code cho hàm Checkwin(

```
Function Checkwin(board, size,symbol,goal,rule)
    int dx[]={0,1,1,1}
    int dy[]={1,0,1,-1}
    for i=0 to size-1
        for j=0 to size-1
            for d=0 to 3
                count =0
                for k=0 to goal-1
                    x=i+dx[d]*k
                    y=j+dy[d]*k
                    if x,y out of board
                        break
                    if board[x][y] != symbol
                        break
                    count++
                if count == goal
                    Function head1=isEmptyhead(board,size,i-dx[d],j-dy[d],symbol)
                    Function head2=isEmptyhead(board,size,i-dx[d] *goal,j-dy[y] *goal,symbol)
                    if head1 and head 2(== true)
                        return true
    return false
```

-Để tối ưu việc quét các hướng,e đã sử dụng 2 mảng dx và dy để biểu thị các tọa độ của nó bao gồm:

+(0,1)->giữ nguyên hàng,chỉ thay đổi cột+1:hướng ngang

+(1,0)->giữ nguyên cột,thay đổi hàng +1: hướng dọc

+(1,1)->tăng cả cột lẫn hàng:chéo chính

+(1,-1)->tăng hàng nhưng giảm cột:chéo phụ

x=i+dx[d]*k: x là tọa độ mới,

i là tọa độ gốc ban đầu,

dx[d] là đang xét theo hướng nào,

k là số bước tăng.

x1=i-dx[d] ,y1=j-dy[d] :đây là tọa độ head1 tức là đầu trên mình xét,mình xem tọa độ này có năm ở trong bảng trò chơi hay ko,và xem tọa độ này có cùng với kí hiệu mà mình đang xét hay ko

x2=i-dx[d]*goal ,y2=j-dx[d]*goal :đây là tọa độ head2 và là đầu dưới của hướng mình xét,vì cần nhảy qua hết goal thì mới đến được ô cuối của chuỗi,nên phải nhân với goal để tính ô cuối cùng,xem tọa độ này có năm ở trong bảng trò chơi hay ko,và xem tọa độ này có cùng với kí hiệu mà mình đang xét hay ko

- Với hàm checkdraw() thì e sẽ chạy 2 vòng for lồng nhau để duyệt hết các ô trên bảng,nếu tại ô mình xét mà vẫn còn kí hiệu '-' này thì có nghĩa bảng còn trống và cần phải điền tiếp,nếu bảng đã full các symbol thì nó sẽ chạy qua hàm checkwin(), từ hàm checkwin() nếu chạy hết mà không trả kết quả ai thắng,nó sẽ trả false và sẽ chạy qua hàm checkdraw() rồi trả true->Hòa - Điều kiện để 1 trận hòa chính là tất cả các ô phải full symbol{'X','O'} mà chưa có ai thắng

4.Game Engine

- Game Engine điều phối giữa người và bot bằng currentPlayer(currentPlayer sẽ nhận 2 giá trị 0 và 1,tức là người chơi 1 và người chơi 2) và gameSetup

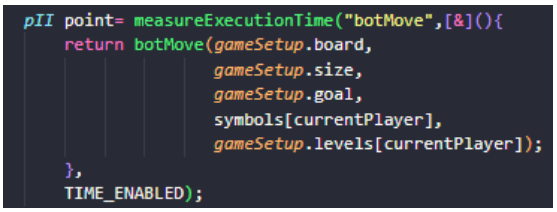
```
bool isbot=(gameSetup.mode==GameMode::EVE)||(gameSetup.mode==GameMode::PVE&& currentPlayer==1);
```

+ khi isBot là true->gọi botMove và thực hiện nước đi
+ nếu false->getplayermove :chờ người nhập nhập nước đi

PVE: Khi vòng đầu tiên qua tức là lần 1 người chơi nhập(currentPlayer=0) ,lần 2 Bot nhập(currentPlayer=1), thì sang vòng thứ 2:currentPlayer sẽ đc reset lại về 0 qua lệnh `currentPlayer=1-currentPlayer;` ,và sẽ bắt đầu lại người chơi 1 nhập rồi tiếp tục Bot nhập và cứ thế vòng lặp diễn ra.

- Sau mỗi nước đi,thì GameEngine sẽ lưu vào 2 hàm checkwin và checkdraw
    + nếu Checkwin = true->lưu kết quả vào GameResults->trả return results-> gọi hàm endgame()
    + nếu Checkdraw=true-> winner=-1-> return results->gọi hàm endgame()

5.Sử dụng GameHelper

- 
```
pII point= measureExecutionTime("botMove",[&](){
    return botMove(gameSetup.board,
                   gameSetup.size,
                   gameSetup.goal,
                   symbols[currentPlayer],
                   gameSetup.levels[currentPlayer]);
    },
    TIME_ENABLED);
```
sử dụng hàm measureExecutionTime để đo thời gian mà bot chạy,nếu mình chọn Bot là Easy thì nó sẽ trả kết quả nhanh,bởi vì phải chọn ngẫu nhiên,còn nếu 2 levels còn lại thì time nó sẽ chạy lâu hơn 1 chút so với EASY
Khi isBot=true thì hàm này bắt đầu chạy đếm thời gian->botmove chạy->trả row,col-> kết thúc đếm thời gian->sau đó lại ghi vào log

3.Kết quả thực nghiệm

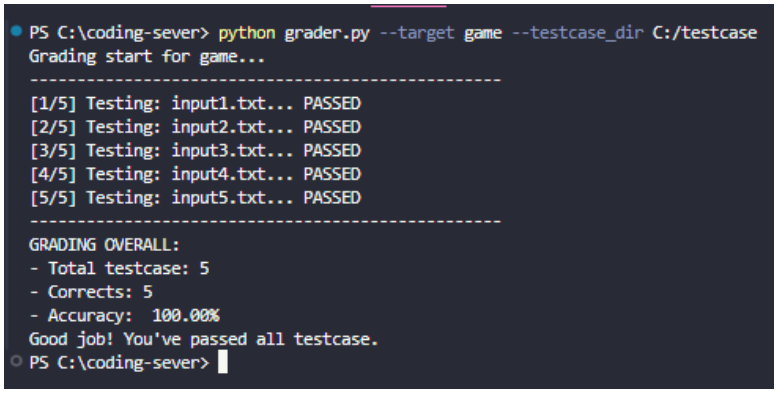
Bảng 5x5:

Menu thiết lập:
```
Size Input(NxN,3 <= N <= BOARD_N_MAX): 5
Goal Input(3 - 5, goal <= size): 3
(1) PvP | (2) PvE | (3) EvE: 2
(1) EASY | (2) MEDIUM | (3) HARD: 1
```

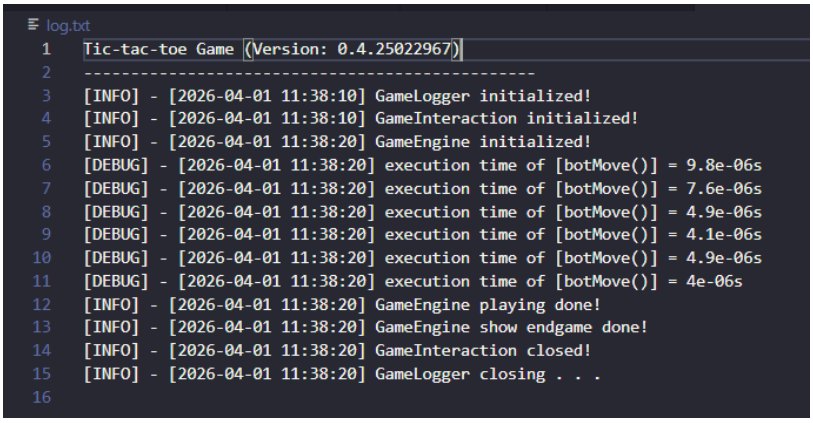
Bàn chơi:
```
  0 1 2 3 4
 -----------
0 |- - - - -
1 |- - - - -
2 |- - - - -
3 |- - - - -
4 |- - - - -
```

Thông báo nước đi:
```
Player 1's turn
Move (row col):
```

Báo cáo Grader:



Trích dẫn log:



4.Tổng kết bài làm

+ Về phong cách lập trình:

Trong quá trình thực hiện project, e đã sử dụng phong cách lập trình thủ tục với cấu trúc chính là startGame() -> playGame() -> endGame(), bên trong được chia nhỏ thành các hàm như validateInput(), selectSize(), selectGameMode(), botMove()...

-Điểm mạnh: có luồng chạy rõ ràng, dễ đọc và dễ debug từng bước. Các hàm nhỏ có nhiệm vụ cụ thể nên có thể kiểm tra riêng lẻ dễ dàng.

-Điểm yếu:khi game phức tạp hơn. Dữ liệu như gameSetup, board, levels phải truyền qua lại giữa nhiều hàm bằng con trỏ và tham số, dễ gây nhầm lẫn. Ngoài ra các hàm bắt đầu phụ thuộc nhau - VD: selectGoal cần biết size, selectBotLevel cần biết mode - khiến việc sửa một chỗ có thể ảnh hưởng nhiều chỗ khác.

-Về khả năng bảo trì:khi game mở rộng thêm tính năng thì sẽ khó kiểm soát hơn.

+ Về giao diện người dùng:

Khi dùng Terminal làm màn hình trò chơi thì e thấy thú vị,vì lần đầu được chơi dạng nhập tọa độ như này,cũng có thể game này là do tự tay em code nên e khá thích thú,nó có điểm lợi là giúp e quan sát rõ output,chạy chỉ cần có Terminal là chạy đc rồi nên khá đơn giản,phụ hợp với lập trình viên.Còn về mặt khó khăn thì khi dùng Terminal giao diện sẽ kém trực quan(người dùng phải nhớ lệnh,tọa độ),khó có thể hiện thị animation.Để so với GUI,thì GUI cho đồ họa đẹp hơn,có animation,âm thanh,input bằng cách click bằng chuột để chọn ô nên sẽ tiện lợi hơn và ai cũng dùng đc,ko cần hướng dẫn.

-Về tổ chức mã nguồn: Việc tập trung toàn bộ vào 1 file thì:

Điểm mạnh:

+ Dễ dàng compile chỉ với 1 lệnh
+ Chia sẻ dễ dàng chỉ với 1 file
+ Ko cần quản lí nhiều file 1 lúc
+ Phù hợp với các project nhỏ,1 người làm

Điểm yếu:

+ Nếu file quá dài thì rất khó tìm hàm cần sửa
+ Sửa 1 chỗ có thể ảnh hưởng tới chỗ khác
+ Khó tái sử dụng cho các project khác
+ Nếu nhiều người dùng cùng 1 file thì rất dễ nhầm lẫn,khó sửa

Cách tổ chức khác: tách thành nhiều file riêng lẻ cho từng chức năng chính của game,code riêng từng file rồi ghép lại làm 1

-Bài học rút ra: Khó khăn nhất với e đối với bài tập này đó chính là phần checkWin và playGame,với e người mới vào lập trình thì phải hiểu từng code rồi mới code được cho đúng và chạy được khá là lâu

5.Ghi chú sử dụng AI

+ Tên công cụ AI đã sử dụng:Claude
+ Mục đích sử dụng:khi làm những hàm khó và khá phức tạp,e sẽ bảo AI gợi ý trước các key để làm VD(như hàm checkWin: AI đã gợi ý e dùng 2 mảng dx[] và dy[] để check 4 hướng) cứ mỗi lần bí là nó sẽ gợi ý 1 ít để hoàn thiện,sau khi hoàn thiện những thứ còn lại theo ý tưởng thì e sẽ dùng AI check qua 1 lượt để rà những lỗi sai,và cứ thế e dần hoàn thiện.
+ Mức độ sử dụng: 30-35%